# MongoDB Design

# Our current DB design

```
{
  prod_name: …,
  prod_price: …,
  prod_man: {
    man_name: …,
    man_address:
…
  }
}
```

Duplicates manufacturer info
when it makes multiple products.

# A BUNCH OF OPTIONS

# One collection, no embedding

```
{
  prod_name: …,
  prod_price:
…,
  man_name: …,
  man_address:
…
}
```

No.  Doesn't capture semantic structure.

# One collection, embedding

```
{
  prod_name: …,
  prod_price: …,
  prod_man: {
    man_name: …,
    man_address:
…
  }
}
```

```
{
  man_name: …,
  man_address: …,
  man_prods: [
    {
      prod_name:
…,
      prod_price:
…
    },
    …
  ]
}
```

# Two collections, referencing

```
{
  _id: 'M1',
  man_name: …,
  man_address:
…
}

{
  _id: 'P1',
  prod_name: …,
  prod_price:
…,
  prod_man: 'M1'
}
```

```
{
  _id: 'M1',
  man_name: …,
  man_address: …
  man_prods: ['P1',
…]
}

{
  _id: 'P1',
  prod_name: …,
  prod_price: …
}
```

```
{
  _id: 'M1',
  man_name: …,
  man_address: …
  man_prods: ['P1',
…]
}

{
  _id: 'P1',
  prod_name: …,
  prod_price: …,
  prod_man: 'M1'
}
```

# Benefits of embedding

- Efficient lookups
- No joins
- Atomicity
  - Transaction granularity = document
- Locality

# Benefits of referencing

- Avoids redundancy
  - Efficient writes
  - Consistency
- Smaller documents (esp. with high arity)
  - Avoids 16MB document limit
  - Less I/O

# Rules of thumb

- Usage determines design
  - Which data do we search on most?
- Prefer embedding unless this causes problems

# Summary – document classes

Which stand alone, which are embedded?

- Guidelines for stand-alone classes:
  - Typically, few
  - Many queries of this data
  - Many updates of this data (costly to replicate)
  - Data referenced by multiple other classes
  - Reference data – e.g., set of all students

- See:  MongoDB Schema Design Best Practices

# Summary – documents

- Describe typical document of each class.
  - Attribute : type of value
  - Is value an array or subdocument?
  - Required vs. optional attributes
- Typically, under max document size.